Subir o Astro (app + backend) acessível pela internet — guia de bolso

Guia pra levantar a **apresentação** (slide 10 com o app embutido) e o **app em tela cheia** ligados a um **backend de verdade**, acessível de qualquer lugar, com o PC servidor ligado. Serve tanto pro servidor principal quanto pra **subir outro na faculdade** se der problema.

> **Plano B rápido:** se só precisa de um servidor funcionando pra demo, siga o **CAMINHO RÁPIDO** abaixo (modo teste da facial — a câmera abre e sempre aprova). O reconhecimento facial **de verdade** está na seção 9, é opcional e demora mais.

>

> Cada servidor é **independente**: banco próprio e URLs próprias. As URLs do Cloudflare grátis **mudam a cada vez** que os túneis sobem.

CAMINHO RÁPIDO (modo demo, ~10 min)

Precisa de: **Python 3.12** (NÃO 3.14 — ver seção de problemas), **Node 18+**, **git**. Abra o **Git Bash** e rode, um bloco por vez:

```
# 1) Baixar só o necessário
git clone --no-checkout --depth 1 --branch apresentacao-interativa \
  --filter=blob:none https://github.com/alefkaue/payment-platform.git astro && cd astro
git sparse-checkout init --cone
git sparse-checkout set apresentacao backend app
git checkout

# 2) Backend: venv + dependências LEVES (sem TensorFlow)
cd backend
py -3.12 -m venv .venv
.venv/Scripts/python.exe -m pip install --upgrade pip
.venv/Scripts/python.exe -m pip install fastapi==0.115.0 "uvicorn[standard]==0.30.6" \
  pydantic==2.9.2 pydantic-settings==2.5.2 email-validator==2.2.0 "passlib[bcrypt]==1.7.4" \
  bcrypt==4.0.1 PyJWT==2.9.0 "cryptography>=43" "sqlalchemy>=2.0.30" python-dotenv \
  "alembic>=1.13.2" "httpx>=0.27"

# 3) Segredos + .env (modo teste da facial)
JWT=$(.venv/Scripts/python.exe -c "import secrets;print(secrets.token_urlsafe(48))")
EMB=$(.venv/Scripts/python.exe -c "from cryptography.fernet import Fernet;print(Fernet.generate_key().decode())")
cat > .env <<EOF
AMBIENTE=desenvolvimento
JWT_SECRET=$JWT
EMBEDDING_KEY=$EMB
ADMIN_EMAIL=admin@payflow.com.br
ADMIN_SENHA=payflow-admin-dev
BIOMETRIA_STUB=1
CNPJ_PROVEDOR=stub
CORS_ORIGINS=http://localhost:8777
EOF

# 4) Criar o banco (SQLite) e subir o backend
.venv/Scripts/python.exe -m alembic upgrade head
.venv/Scripts/python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Deixe esse terminal aberto. Teste noutra aba: `curl http://localhost:8000/` deve responder com `"sqlite"`.
Admin: `admin@payflow.com.br` / `payflow-admin-dev`.

5) Túnel do backend → descobrir a URL pública da API

Num **novo** terminal, baixe o cloudflared uma vez e suba o túnel:

```
curl -L -o cloudflared.exe \
  https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-windows-amd64.exe
./cloudflared.exe tunnel --no-autoupdate --url http://localhost:8000
```

Ele imprime uma URL tipo `https://xxxx.trycloudflare.com`. **Essa é a URL_BACKEND.** Deixe rodando.

6) Buildar o app apontando pro backend e colar na apresentação

Noutro terminal (troque `URL_BACKEND` pela URL real do passo 5):

```
cd astro/app
echo "VITE_API_URL=URL_BACKEND" > .env
npm ci
npm run build
node scripts/assemble-www.mjs            # gera app/www/

cd ..
rm -rf apresentacao/assets apresentacao/welcome
cp -r app/www/assets  apresentacao/assets
cp -r app/www/welcome apresentacao/welcome
cp app/www/index.html apresentacao/index.html
cp app/www/robots.txt app/www/favicon.ico app/www/favicon.svg \
   app/www/apple-touch-icon.png app/www/icon-192.png app/www/icon-512.png apresentacao/
```

7) Servir a apresentação + túnel dela

```
cd apresentacao
py servir.py                             # serve em http://localhost:8777 (fallback SPA)
```

Noutro terminal:

```
./cloudflared.exe tunnel --no-autoupdate --url http://localhost:8777
```

A URL impressa é a **URL_APRESENTACAO**. Abra ela na faculdade:

- **App em tela cheia:** `URL_APRESENTACAO/`
- **Apresentação (slide 10):** `URL_APRESENTACAO/Astro%20Apresentacao.dc.html`

8) Liberar o CORS do backend pra apresentação

No `astro/backend/.env`, troque a linha do CORS por:

```
CORS_ORIGINS=URL_APRESENTACAO,http://localhost:8777
```

e **reinicie o uvicorn** (Ctrl+C no terminal do backend e suba de novo). Sem isso o app embutido não chama a API (erro de CORS no console do navegador). **Pronto.**

---

9) OPCIONAL — reconhecimento facial DE VERDADE

No caminho rápido a facial abre a câmera mas **aprova qualquer rosto** (ótimo pra demo). Pra ela conferir o rosto de verdade:

```
cd astro/backend
# instala as libs pesadas de IA (uns 500 MB+; demora)
.venv/Scripts/python.exe -m pip install "deepface[tensorflow]>=0.0.93" "tf-keras>=2.15" \
  opencv-python==4.10.0.84 "numpy>=1.24"

# desliga o modo teste
#   (edite o .env e troque BIOMETRIA_STUB=1 por BIOMETRIA_STUB=0)
```

IMPORTANTE — rode o backend com UTF-8, senão a DeepFace quebra no Windows (ver problemas abaixo):

```
PYTHONUTF8=1 .venv/Scripts/python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Na 1ª verificação a DeepFace baixa os modelos (Facenet + anti-spoofing). Pra adiantar: `PYTHONUTF8=1 .venv/Scripts/python.exe -c "from deepface import DeepFace; DeepFace.build_model('Facenet')"`. Cadastre **o seu** rosto com boa luz — depois o sistema rejeita outras pessoas.

Problemas comuns (o que a gente já apanhou)

• **A facial dá erro 500 /** `charmap codec can't encode` → rode o uvicorn com `PYTHONUTF8=1` na frente (a DeepFace imprime um emoji que o console Windows não aceita). Esse é o pulo do gato.

• **TensorFlow não instala / erro de wheel** → você está no **Python 3.14**. O TF só vai até o 3.12/3.13. Crie o venv com `py -3.12 -m venv .venv`.

• **"Could not start video source" / câmera não abre** → o Chrome está usando uma câmera virtual parada (ex.: OBS Virtual Camera). Vá em `chrome://settings/content/camera` e escolha a **webcam real**. Precisa ser **HTTPS** (o túnel já é) ou localhost pra liberar a câmera.

• **"Cloudflare Tunnel error" / 1033 na página** → é um blip do túnel grátis. **F5** resolve; a URL não muda enquanto o `cloudflared` continuar rodando.

• **App abre mas em "modo demonstração" (não salva nada)** → faltou o `VITE_API_URL` no build, ou você não copiou o `www/` novo pra dentro de `apresentacao/`. Refaça os passos 6.

• **Erro de CORS no console** → a `URL_APRESENTACAO` não está no `CORS_ORIGINS` do backend, ou você esqueceu de reiniciar o uvicorn depois de editar o `.env` (passo 8).

Parar / suspender (no servidor principal, em casa)

• **Parar tudo agora:** rode `stop-astro.ps1` (na raiz do projeto) — ele mata as portas 8000/8777, o cloudflared, e **suspende** o PC.

• **Desligar sozinho num horário:** há uma tarefa do Windows (`AstroStop2300`) que roda esse script às 23:00. Pra recriar/ajustar: `` schtasks /create /tn "AstroStop2300" /sc once /st 23:00 /f ^ /tr "powershell -NoProfile -ExecutionPolicy Bypass -File C:\caminho\stop-astro.ps1" ``

• **Não deixar dormir antes da hora:** `powercfg /change standby-timeout-ac 0` (e mantenha o notebook **na tomada**). O `stop-astro.ps1` restaura 30 min ao suspender.

Avisos

• Isto expõe um backend (demo) na internet pública. URL obscura e **temporária** — suba só durante os testes e desligue depois.

• Os processos (backend, servidor da apresentação, túneis) morrem se você **fechar o terminal/sessão** que os iniciou. Mantenha abertos e o PC ligado.

• **Reboot derruba tudo e não volta sozinho** (e a URL muda). Pra sobreviver a reboot com URL fixa, seria preciso um túnel nomeado com conta Cloudflare.

• Voltar a apresentação ao build demo original: `git checkout -- apresentacao` (ou restaure `apresentacao/index.demo.html.bak`).